# Reinforcement Learning for Intelligent Traffic Signal Optimization

## A Comprehensive Technical Reference: Microscopic Simulation, Tabular Q-Learning, and Deep Q-Networks (DQN)

**Institution:** Indian Institute of Information Technology, Nagpur (IIITN)
**Mentor:** Dr. Rashmi Pandhare | **Contributors:** Saurabh Kumar • Darshan Tate • Sai Chandra Mouli
**Environment:** Eclipse SUMO (Simulation of Urban MObility) & TraCI (Traffic Control Interface) Python API

**Executive Overview:** Urban traffic intersections governed by static, pre-timed controllers suffer from systemic inefficiencies due to an inability to adapt to stochastic, non-linear arrival rates. This work establishes a closed-loop cyber-physical framework combining microscopic simulation (SUMO), online sensors (induction loop detectors), and Reinforcement Learning (RL) agents. We benchmark a Fixed-Time Static Controller against a discrete Tabular Q-Learning Agent and a Deep Q-Network (DQN) Agent across standard Intelligent Transportation Systems (ITS) delay, queue length, and throughput metrics.

## 1. System Architecture & Closed-Loop Simulation

The simulation ecosystem couples an external agentic controller in Python with Eclipse SUMO through the TraCI socket interface. Every discrete time-step ($\Delta t = 1.0$ second), state feedback is polled, a decision is actuated, safety constraints are evaluated, and reward feedback is computed.

**The 4 Operational Phases per Step:**

1. **Observation Collection:** The simulation queries 6 induction loop detectors situated on Eastbound (Node1_2_EB_0, 1, 2) and Southbound (Node2_7_SB_0, 1, 2) incoming approaches, yielding queue counts along with the active signal phase index.
2. **Policy Evaluation:** The observation vector is passed into the RL agent. An $\varepsilon$-greedy action selection policy picks whether to preserve green time (Action 0) or transition to the subsequent phase (Action 1).
3. **Safety Actuation:** If an action switch is demanded, a minimum green time constraint (MIN_GREEN_STEPS = 5s) prevents unsafe oscillation and ensures vehicular momentum before triggering yellow clearance (3s).
4. **Reward Feedback & Value Update:** SUMO steps forward. The agent measures the resulting system queue backlog, generates reward R, and updates its value function via temporal difference learning.

## 2. Mathematical Formulation (MDP Framework)

Traffic signal optimization is modeled as a Markov Decision Process (MDP) tuple $\langle S, A, P, R, \gamma \rangle$:

**State Space ($S \in \Re^7$):**

The state vector is a 7-dimensional representation: $S_t = [q_0, q_1, q_2, q_3, q_4, q_5, \phi]$, where $q_{0..2}$ represent vehicles queued on the Eastbound lanes, $q_{3..5}$ represent vehicles on Southbound lanes, and $\phi$ represents the discrete traffic signal phase ID at junction Node2.

**Action Space ($A \in \{0, 1\}$):**

- Action 0: Maintain current phase (Keep Green active).
- Action 1: Cycle to next signal phase (Initiate Yellow clearance then switch).

**Reward Function ($R_t$):**

The reward is formulated as the negative sum of vehicular queue lengths across all monitored approaches:
$R_t = -\sum_{i=0}^{5} Queue_i(t)$
Because $R_t \leq 0$, maximizing expected return is mathematically equivalent to minimizing cumulative vehicular queue delay across the intersection.

## 3. Tabular Q-Learning vs. Deep Q-Network (DQN)

While both algorithms seek to approximate the optimal action-value function $Q^*(s, a) = \max_\pi E[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t=s, A_t=a]$, they employ fundamentally distinct computational structures:

| Dimension | Tabular Q-Learning | Deep Q-Network (DQN) |
|---|---|---|
| **Core Data Structure** | Discrete lookup table / Hash map (Python dict) | Deep Feedforward Neural Network (MLP) |
| **Update Equation** | $Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ | $Loss = (Q(s, a; \theta) - [R + \gamma \max_{a'} Q(s', a'; \theta)])^2$ Gradient descent updates weights $\theta$ |
| **Generalization Ability** | **Zero generalization:** Every distinct tuple is a novel state requiring independent exploration. | **High generalization:** Continuous feature spaces allow the network to interpolate across unseen queue combinations. |
| **State Space Scalability** | Suffers from the *Curse of Dimensionality*. Memory scales exponentially: $\|Q\| \propto \|S\| \times \|A\|$. | Highly scalable. Memory is bounded by fixed weight matrices $\theta$ regardless of state space volume. |
| **Inference Latency** | Microseconds (O(1) memory lookup). | Milliseconds (Matrix tensor multiplications). |
| **Stability & Convergence** | Proven guaranteed convergence under Robbins-Monro conditions. | Non-linear function approximation can oscillate without target networks or replay memory. |

# 4. The 3-Way Showdown: Static vs. Q-Learning vs. DQN

Each controller was evaluated over an identical 3000-step simulation episode with high vehicular density. Below are the verified experimental metrics:

| ITS Performance Metric | Static Baseline | Tabular Q-Learning | Deep Q-Network | Improvement / Impact |
|---|---|---|---|---|
| **Total System Delay (veh-s)** | 23,301.00 | **14,344.00** | 14,966.00 | **38.4% reduction** (Q-Learning leads) |
| **Average Queue Length (veh)** | 7.77 | **4.78** | 4.99 | **38.5% queue reduction** |
| **Max Queue Peak (veh)** | 12 | 12 | **11** | DQN caps extreme congestion spikes |
| **Throughput (Vehicles Cleared)** | 1,929 | **2,368** | 2,303 | **+22.7% more vehicles** cleared |
| **Mean Travel Time / Vehicle (s)** | 287.54 | **123.61** | 130.08 | **57.0% faster transit** time |
| **90th Percentile Travel Time (s)** | 409.00 | **172.00** | 191.00 | **58.0% reduction** in tail latency |
| **Maximum Travel Time (Worst-Case)** | 1,332.00 s | 1,325.00 s | **973.00 s** | **DQN wins:** 27.0% lower worst-case |

# 5. Deep Behavioral Analysis of Each Model

### 1. The Static Baseline: Why Pre-Timed Signals Fail

The Static controller cycles predictably through fixed durations (42s green, 3s yellow). When vehicle arrivals are uneven (e.g., a burst of 15 cars arriving Eastbound while Southbound is completely empty), the static controller forces Eastbound vehicles to idle at red while green light is squandered on empty pavement. This leads to runaway delays (23,301 veh-s) and severe queue spikes up to 12 vehicles.

### 2. Tabular Q-Learning: Aggressive, Opportunistic Optimization

Tabular Q-Learning learns exact state transitions rapidly on single intersections where states repeat frequently. When queues build up, the agent triggers aggressive phase shifts as soon as minimum green permits, resulting in the highest vehicle throughput (2,368 cleared) and lowest average delay (14,344 veh-s). However, its maximum travel time remains high (1,325s) because rare queue combinations that the table hasn't encountered frequently will be treated greedily or under-explored.

### 3. Deep Q-Network: Smooth Generalization & Worst-Case Protection

DQN does not memorize states; instead, its 24×24 neural layers learn a smooth manifold of traffic pressure. It recognizes that 6 cars on lane 1 and 4 on lane 2 is functionally identical to 5 cars on lane 1 and 5 on lane 2. This prevents erratic switching and suppresses queue oscillations (queue length stays smoothly in the 3–5 vehicle band). Crucially, DQN achieves a maximum travel time of **973s** (vs. 1332s in Static), providing the most resilient performance against peak gridlock.

# 6. Key Takeaways for Presentation & Defense (Viva)

**Q: Why did Q-Learning have slightly higher throughput than DQN, but DQN had a better max travel time?**

A: Tabular Q-Learning takes opportunistic, aggressive switches tailored to frequently seen states, maximizing average flow. However, DQN's neural generalization prevents extreme outlier queues from ever compounding, making it superior at handling worst-case congestion spikes.

**Q: Why not use a reward based purely on vehicle throughput instead of queue length?**

A: Throughput is a delayed, sparse reward (only awarded when a vehicle finishes its route). Queue length is a dense, immediate signal observable at every single second, allowing stable Temporal Difference learning.

**Q: How can this system be deployed in the physical world?**

A: The Python controller can execute on an edge compute unit (e.g., NVIDIA Jetson Nano or Raspberry Pi 4) connected via NEMA TS2 / 2070 traffic cabinet interfaces, replacing SUMO induction loops with YOLO-based computer vision detector feeds from municipal CCTV cameras.